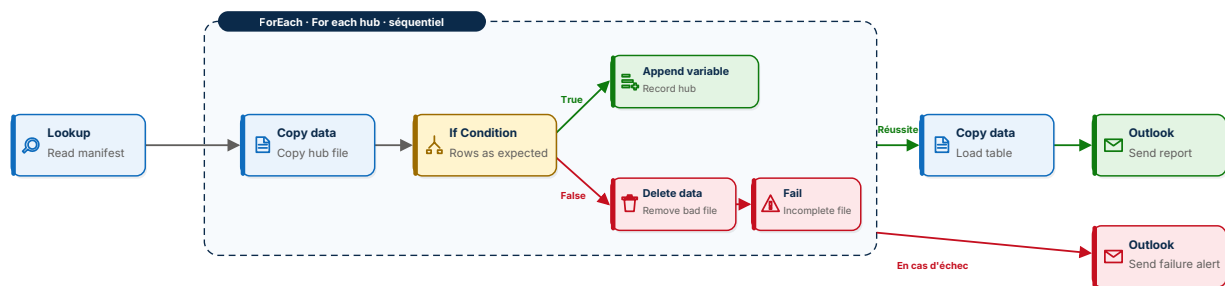


Pipeline Microsoft Fabric : lire, boucler, vérifier, arrêter

Un manifeste liste les fichiers du jour. Le pipeline les lit, les copie un par un, vérifie chacun, retire tout fichier incomplet, reconstruit la table Delta et vous envoie le compte rendu. Dix activités, aucune ligne de code, et un pipeline qui préfère finir en échec plutôt que de charger une donnée fausse.



PL_Deliveries_Express. Fil vert : la boucle a réussi, on consolide et on prévient. Fil rouge : un fichier a été refusé, on alerte.

10
activités

8
types d'activité

0
ligne de code

0
doublon à la relance

7
pages

AUTEUR

Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS

Un espace de travail Fabric sur une capacité, une boîte Outlook Microsoft 365

FICHIERS DU PROJET

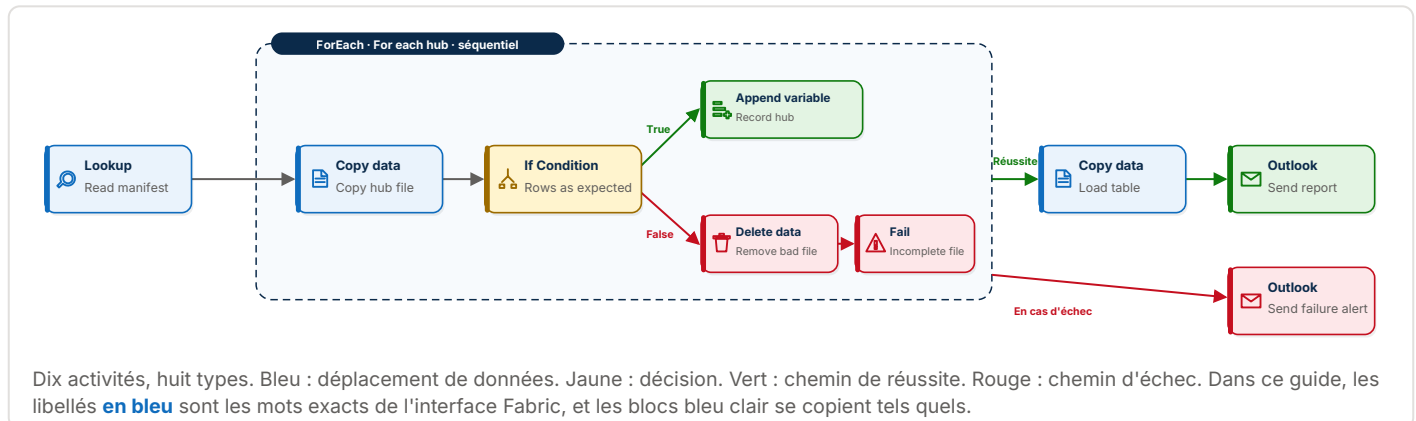
github.com/abdoulhamiddiallo/fabric-pipeline-express-2

ÉDITION

Octobre 2026

Ce que vous construisez

Trois hubs logistiques, Metz, Nancy et Thionville, déposent chaque jour leur fichier de livraisons. Un manifeste `manifest_<date>.csv` dit quels fichiers attendre et combien de lignes au minimum. Le pipeline lit ce manifeste et traite chaque fichier sans qu'aucun nom ne soit écrit en dur.



Piloté par un manifeste

La liste des fichiers vit dans un CSV, pas dans le pipeline. Un hub de plus demain : une ligne de plus dans le manifeste, rien à modifier dans Fabric.

Chaque fichier répond de lui-même

Les lignes copiées sont comparées au minimum attendu. Un fichier incomplet est retiré de la zone brute et l'exécution finit en échec, table intacte.

Une table toujours juste

La table `deliveries` est reconstruite à chaque exécution depuis la zone brute entière. Relancer une journée ne crée jamais un doublon.

Les dix activités

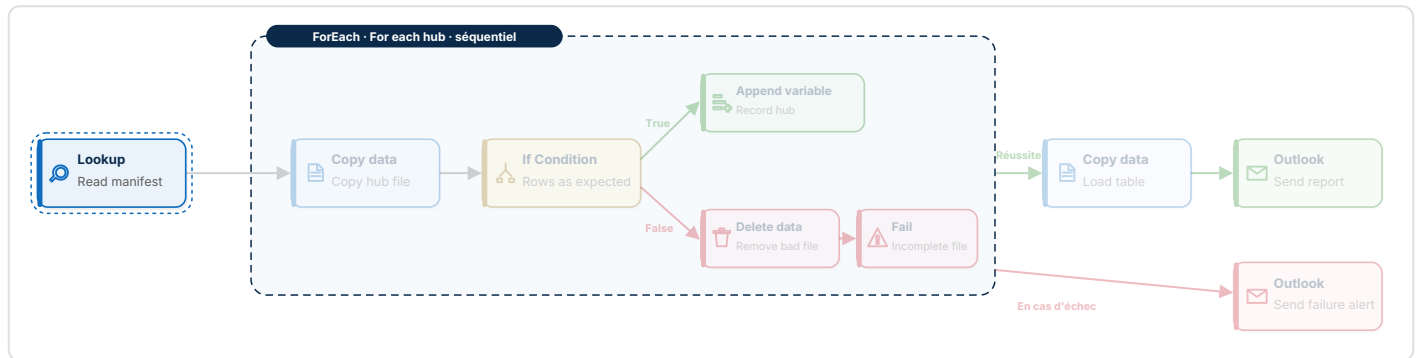
ACTIVITÉ	TYPE	RÔLE
Read manifest	Lookup	Lit <code>manifest_<date>.csv</code> sur GitHub et renvoie ses lignes sous forme de tableau
For each hub	ForEach	Parcourt les lignes du manifeste, en séquentiel
Copy hub file	Copy data	Copie le fichier du hub vers <code>Files/raw/deliveries/load_date=<date>/hub=<hub>/deliveries.csv</code>
Rows as expected	If Condition	Compare les lignes copiées au minimum du manifeste
Record hub	Append variable	Branche True : mémorise « hub : n lignes » pour le compte rendu
Remove bad file	Delete data	Branche False : retire le fichier incomplet de la zone brute
Incomplete file	Fail	Branche False : met l'exécution en échec avec le nom du hub en message
Load table	Copy data	Lit tous les <code>deliveries.csv</code> de la zone brute et remplace la table <code>deliveries</code>
Send report, Send failure alert	Office 365 Outlook	Un e-mail si tout a réussi, un autre si la boucle a échoué

Le jeu de données

JOURNÉE	METZ	NANCY	THONVILLE
2026-10-05	437	384	307
2026-10-06	467	391	305
2026-10-07	448	48	335

Trois journées dans `data/`, un manifeste et un fichier par hub. Le manifeste a trois colonnes : `hub`, `file`, `min_rows` (400, 350, 280). La journée du 7 octobre est volontairement incomplète : le fichier de Nancy ne contient que 48 lignes. C'est elle qui vous servira à voir le pipeline refuser un fichier. Chaque fichier a douze colonnes : `colis`, `hub`, `horodatage`, `chauffeur`, `ville`, `code postal`, `service`, `poids`, `distance`, `durée`, `statut`, `respect du délai`.

Préparer, lire le manifeste, boucler



1 Les objets Fabric

- 1 Créez un lakehouse **LH_Deliveries**, vide, et un pipeline **PL_Deliveries_Express**.
- 2 Fond du canevas, onglet **Paramètres, Nouveau** :

Nom	load_date
Type	String
Valeur par défaut	2026-10-05

- 3 Onglet **Variables, Nouveau** :

Nom	hub_rows
Type	Array (Tableau)
Valeur par défaut	vide

POURQUOI UN ARRAY La boucle y ajoutera un élément par hub ; un seul `join` en fera une phrase pour l'e-mail.

2 La connexion HTTP

Elle se crée depuis la première activité qui en a besoin, le Lookup. Liste **Connexion, Plus**, recherchez **HTTP** :

URL	https://raw.githubusercontent.com/abdoulhamiddiallo/fabric-pipeline-express-2/main/data/
Authentification	Anonyme
Nom	github_deliveries

UNE HABITUDE QUI SAUVE UNE SOIRÉE **Enregistrer** après chaque activité terminée, et vérifier que le point à côté du nom du pipeline a disparu. Un pipeline jamais enregistré n'existe que dans votre navigateur.

3 Read manifest

LOOKUP

- 1 Ajoutez **Rechercher** (Lookup), nom `Read manifest`.
- 2 **Paramètres** : connexion `github_deliveries`, type **HTTP**, **URL relative** en contenu dynamique :

```
manifest_@{pipeline().parameters.load_date}.csv
```

- 3 Format **DelimitedText**, **Première ligne comme en-tête** coché.
- 4 Décochez **Première ligne uniquement** : c'est ce réglage qui renvoie toutes les lignes. **Aperçu des données** : trois lignes, une par hub.

CE QUE RENVOIE L'ACTIVITÉ `output.value`, un tableau de trois éléments ayant les propriétés `hub`, `file`, `min_rows`, les en-têtes du CSV. Les valeurs arrivent en texte.

4 For each hub

FOREACH

- 1 Ajoutez **ForEach**, nom `For each hub`, relié à la sortie verte de `Read manifest`.

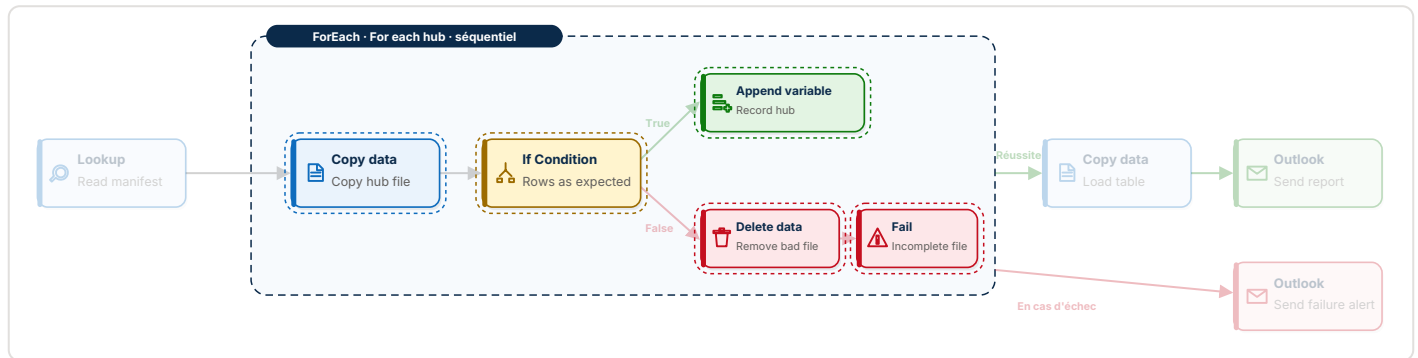
- 2 **Paramètres** : cochez **Séquentiel. Éléments** :

```
@activity('Read manifest').output.value
```

- 3 Cliquez sur le crayon du ForEach : vous entrez dans la boucle, un canevas vide pour l'étape 3. Le fil d'Ariane vous ramène au pipeline.

SÉQUENTIEL, ET NON PARALLÈLE La boucle écrit dans une variable ; en parallèle, trois itérations écriraient en même temps. Dans la boucle, `item()` est la ligne courante : `item().hub`, `item().file`, `item().min_rows`.

À l'intérieur de la boucle : copier, vérifier, décider



5 Copy hub file

COPY DATA

- 1 Dans la boucle, ajoutez **Copier les données**, nom Copy hub file.
- 2 **Source** : connexion github_deliveries, **DelimitedText**, en-tête coché, **URL relative** : `@{item().file}`
- 3 Dépliez **Avancé, Colonnes supplémentaires, Nouveau** : nom load_date, valeur `@pipeline().parameters.load_date`.
- 4 **Destination** : Lakehouse, LH_Deliveries, **Fichiers, DelimitedText**, en-tête coché. Les deux cases du chemin :

```
raw/deliveries/
load_date=@{pipeline().parameters.load_date}/
hub=@{item().hub}
```

```
deliveries.csv
```

POURQUOI LA COLONNE SUPPLÉMENTAIRE Sans elle, un CSV copié vers un CSV identique est un simple transfert de fichier : l'activité ne compte pas les lignes. Avec elle, Fabric lit chaque ligne et renvoie rowsCopied. Bonus : chaque fichier brut porte sa date en colonne.

6 Rows as expected

IF CONDITION

- 1 Ajoutez une **Condition Si**, nom Rows as expected, reliée à la sortie verte du Copy. **Expression** :

```
@greaterOrEquals(activity('Copy hub
file').output.rowsCopied, int(item().min_rows))
```

rowsCopied exclut l'en-tête. min_rows arrive en texte ; int() le convertit.

7 Record hub

APPEND VARIABLE · TRUE

- 1 Cliquez sur le + de la case **True**, choisissez **Ajouter une variable**, nom Record hub. Variable hub_rows, valeur :

```
@concat(item().hub, ' : ', string(activity('Copy
hub file').output.rowsCopied), ' lignes')
```

8 Remove bad file

DELETE DATA · FALSE

- 1 Cliquez sur le + de la case **False**, choisissez **Supprimer les données**, nom Remove bad file.
- 2 **Source** : Lakehouse, LH_Deliveries, **Fichiers**, même dossier daté et même nom deliveries.csv que la destination du Copy.
- 3 Onglet **Paramètres de journalisation** : décochez **Activer la journalisation**.

9 Incomplete file

FAIL · FALSE

- 1 Dans la case False, ajoutez **Échec** (Fail), nom Incomplete file, relié à la sortie verte de Remove bad file.
- 2 **Code d'erreur** 400, **Message d'échec** :

```
@concat('Fichier incomplet pour le hub ',
item().hub)
```

PORTÉE DES EXPRESSIONS La première activité d'une branche voit Copy hub file ; une activité placée derrière une autre, comme Incomplete file, ne le voit plus. D'où un message sans le nombre de lignes, qui reste lisible dans la sortie du Copy.

CE QUE FAIT LA BRANCHE FALSE Le fichier incomplet est retiré avant d'atteindre la table ; la boucle termine les autres hubs, puis est marquée en échec. Rien n'est chargé.

AVANT DE QUITTER LA BOUCLE

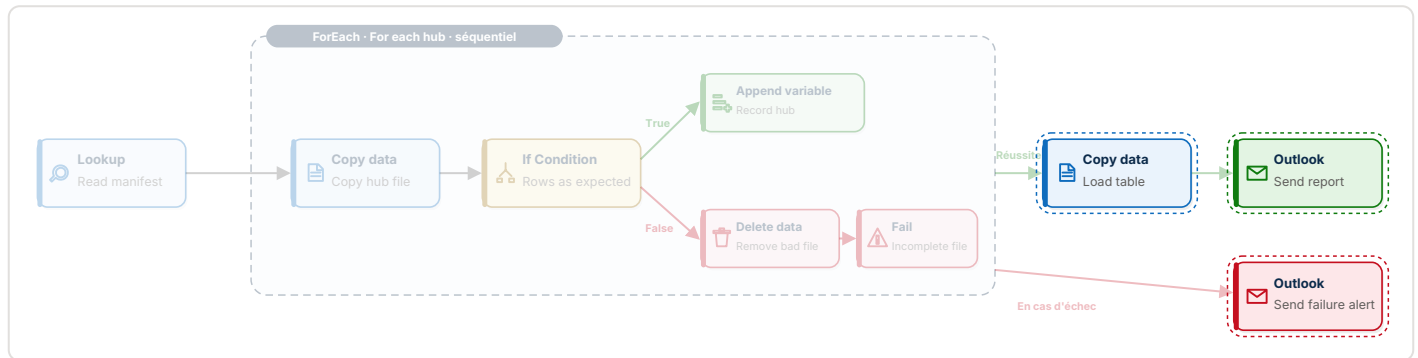
☐ Copy hub file a sa colonne supplémentaire load_date

☐ la condition est reliée à la sortie verte du Copy

☐ Record hub est seul dans True, Remove bad file puis Incomplete file dans False

☐ Enregistrer, le point a disparu

Consolider la table et prévenir



Revenez au canevas principal par le fil d'Ariane ; les trois activités suivantes se posent après la boucle.

10 Load table

COPY DATA

- 1 Ajoutez **Copier les données**, nom **Load table**, relié à la sortie verte de **For each hub**.
- 2 **Source** : **Lakehouse**, **LH_Deliveries**, **Fichiers**. Dans **Type de chemin d'accès au fichier**, choisissez **Chemin d'accès au fichier générique** ; les deux cases changent de nom :

Chemin du dossier générique	raw/deliveries/*/
Nom de fichier générique	deliveries.csv

Format **DelimitedText**, en-tête coché.

- 3 **Avancé**, **Colonnes supplémentaires**, **Nouveau** : nom **source_file**, valeur **\$\$FILEPATH**.
- 4 **Destination** : **Lakehouse**, **LH_Deliveries**, **Tables**, table **deliveries**, action **Remplacer** (Overwrite).
- 5 Laissez **Mappage** vide : il se remplit après la première exécution (étape 5).

TROIS IDÉES DANS UN SEUL COPY Le motif ***/** descend dans les dossiers **load_date=.../hub=...** de toutes les journées. **\$\$FILEPATH** inscrit dans chaque ligne le chemin d'où elle vient. Remplacer reconstruit la table entière : relancer redonne exactement la même table.

11 Send report

OFFICE 365 OUTLOOK

- 1 Ajoutez **E-mail Office 365**, nom **Send report**, relié à la sortie verte de **Load table**. Connectez votre compte, renseignez **À**.

- 2 **Objet**, en contenu dynamique :

```
Livraisons @{pipeline().parameters.load_date} :
@{activity('Load table').output.rowsCopied}
lignes dans la table deliveries
```

- 3 **Corps** : tapez le texte au clavier (le collage n'y fonctionne pas), puis insérez les valeurs par l'éclair, onglet **Fonctions**, sans le @ de tête :

```
Fichiers vérifiés : join(variables('hub_rows'),
'; ')
Table reconstruite : activity('Load
table').output.rowsCopied lignes
```

12 Send failure alert

OFFICE 365 OUTLOOK · EN CAS D'ÉCHEC

- 1 Seconde activité **E-mail Office 365**, nom **Send failure alert**, sur le canevas principal, hors de la boucle.
- 2 Reliez-la à **For each hub** par le connecteur rouge **En cas d'échec** (le X), et non par le vert.
- 3 **Objet** : Pipeline arrêté pour **@{pipeline().parameters.load_date}**. **Corps**, au clavier : « Fichier incomplet, zone brute nettoyée, table intacte. »

DEUX FILS, DEUX FINS PROPRES Fil vert si la boucle réussit, fil rouge si elle échoue : dans les deux cas un e-mail part. **Valider**, puis **Enregistrer**.

Ce que contiendra le lakehouse

LH_Deliveries

Files / raw / deliveries

- load_date=2026-10-05 : METZ 437 · NANCY 384 · THIONVILLE 307
- load_date=2026-10-06 : METZ 467 · NANCY 391 · THIONVILLE 305
- load_date=2026-10-07 : METZ 448 · THIONVILLE 335 · NANCY retiré

Tables / deliveries

reconstruite à chaque exécution : 8 fichiers, 3 074 lignes, colonne **source_file**

La zone brute est la mémoire du projet : chaque fichier accepté y reste, dans un dossier qui porte sa date et son hub. La table n'est qu'une vue consolidée de cette zone, reconstruite à chaque exécution : relancer ne duplique jamais, et le fichier refusé de Nancy n'apparaît nulle part. Un fichier corrigé demain ? Relancez la journée : le Copy le remplace au même emplacement, le contrôle le valide, la table est reconstruite.

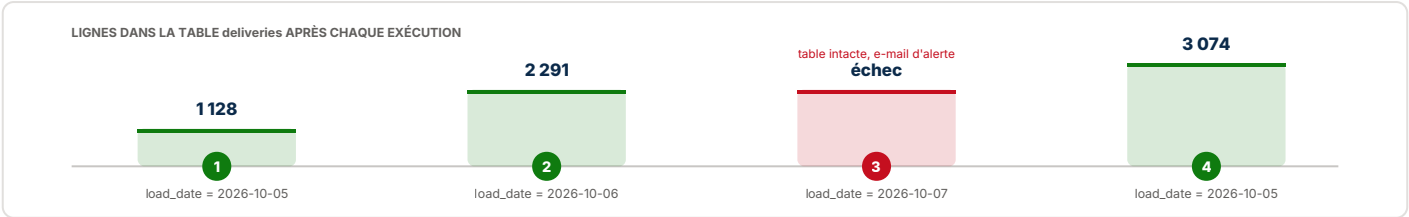
Exécuter et vérifier

OBJECTIF

Quatre exécutions : le chemin normal, l'accumulation, l'échec propre, l'absence de doublon.

RÉSULTAT

Les chiffres ci-dessous, à l'unité près.



Exécution 1 load_date = 2026-10-05

Exécuter, valeur par défaut. Onglet **Sortie** : Read manifest renvoie count = 3 ; les trois Copy hub file donnent 437, 384 et 307 ; Load table donne **1128**. E-mail reçu : « Livraisons 2026-10-05 : 1128 lignes dans la table deliveries ».

MAINTENANT, LES TYPES Dans Load table, onglet **Mappage**, **Importer des schémas** réussit désormais. Passez weight_kg et distance_km en Double, delivery_minutes en Int32, delivery_datetime en DateTime, enregistrez : la prochaine exécution remplace la table avec les bons types.

Exécution 2 load_date = 2026-10-06

Les Copy donnent 467, 391 et 305. Load table donne **2 291** : les deux journées, puisque la table est reconstruite depuis toute la zone brute.

Exécution 3 load_date = 2026-10-07, la journée incomplète

Metz passe (448). Nancy copie 48 lignes, la condition tombe dans False : Remove bad file retire le fichier, Incomplete file lève « Fichier incomplet pour le hub NANCY ». La boucle termine Thionville (335, conservé) puis est marquée en échec ; Load table et Send report sont ignorés, Send failure alert part. L'exécution est marquée **Échec** : c'est le comportement voulu, et la table n'a pas bougé.

Exécution 4 load_date = 2026-10-05, à nouveau

Load table donne **3 074** : les deux journées complètes plus Metz et Thionville du 7 octobre, qui avaient passé le contrôle. Rien du fichier incomplet. Relancez encore : toujours 3 074.

Vérifier dans le point de terminaison SQL

Ouvrez LH_Deliveries, basculez sur **Point de terminaison d'analyse SQL**, nouvelle requête :

```
SELECT hub, COUNT(*) AS lignes,
SUM(CASE WHEN status = 'delivered' THEN 1 ELSE 0
END) AS livrees,
SUM(CASE WHEN on_time = 'yes' THEN 1 ELSE 0 END) AS
a_l_heure,
COUNT(DISTINCT source_file) AS fichiers
FROM deliveries GROUP BY hub ORDER BY hub;
```

HUB	LIGNES	LIVREES	A_L_HEURE	FICHIERS
METZ	1 352	1 221	989	3
NANCY	775	703	578	2
THIONVILLE	947	858	723	3

3 074

lignes dans la table

2 782

livraisons effectuées, 90,5 %

2 290

à l'heure, 74,5 %

8 / 9

fichiers acceptés, 1 refusé

CE QUE CES CHIFFRES PROUVENT Aucun nom de fichier dans le pipeline, chaque fichier vérifié un par un, un fichier incomplet refusé sans pénaliser les autres (Nancy n'a que 2 fichiers), et aucun doublon après relance.

Expressions et débannage

Les expressions du pipeline

OÙ	EXPRESSION
Read manifest, URL relative	<code>manifest_@{pipeline().parameters.load_date}.csv</code>
For each hub, Éléments	<code>@activity('Read manifest').output.value</code>
Copy hub file, URL relative	<code>@{item().file}</code>
Copy hub file, colonne supplémentaire	<code>load_date = @pipeline().parameters.load_date</code>
Copy hub file, dossier de destination	<code>raw/deliveries/load_date=@{pipeline().parameters.load_date}/hub=@{item().hub}</code>
Rows as expected, Expression	<code>@greaterOrEquals(activity('Copy hub file').output.rowsCopied, int(item().min_rows))</code>
Record hub, Valeur	<code>@concat(item().hub, ' : ', string(activity('Copy hub file').output.rowsCopied), ' lignes')</code>
Incomplete file, Message	<code>@concat('Fichier incomplet pour le hub ', item().hub)</code>
Load table, chemin générique	<code>raw/deliveries/*/* et deliveries.csv ; colonne source_file = \$\$FILEPATH</code>
Send report, Objet	<code>Livraisons @{pipeline().parameters.load_date} : @{activity('Load table').output.rowsCopied} lignes dans la table deliveries</code>
Send report, Corps	<code>join(variables('hub_rows'), ' ; ') inséré par l'onglet Fonctions</code>

DEUX FAÇONS D'ÉCRIRE Dans un champ texte (URL relative, chemin, objet), l'expression s'insère entre `@{ }` au milieu du texte. Dans un champ expression (Éléments, Expression, Valeur d'une variable, Message d'échec), elle commence par `@` et occupe tout le champ.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Le Lookup ne renvoie qu'une ligne	Première ligne uniquement est resté coché. Décochez-le.
« property 'rowsCopied' doesn't exist »	Le Copy a transféré le fichier sans lire les lignes. Ajoutez la colonne supplémentaire <code>load_date</code> dans la source de Copy hub file.
« Impossible de référencer la sortie de l'activité... pas un ancêtre »	Soit le nom diffère de celui de l'activité (casse, espaces), soit l'activité est placée derrière une autre dans une branche : utilisez une expression sans cette référence.
L'onglet Journalisation du Delete affiche un « 1 » rouge	La journalisation est activée par défaut et exige un compte. Décochez Activer la journalisation .
Load table : « PathNotFound ... raw/deliveries/deliveries.csv »	Le type de chemin est resté « Chemin d'accès au fichier ». Passez en chemin générique avec <code>raw/deliveries/*/*</code> , enregistrez, relancez.
« Importer des schémas » échoue	La zone brute est encore vide. Exécutez une première fois, puis importez.
Le texte ne se colle pas dans le corps de l'e-mail	Tapez-le au clavier ; les valeurs dynamiques passent par l'éclair, onglet Fonctions.
Send failure alert part à chaque itération	Il est dans la boucle. Placez-le sur le canevas principal, relié au connecteur rouge de For each hub.
Valider et Exécuter restent grisés, un rond tourne	Enregistrement en cours. Patientez, fermez le panneau du bas ; ne rechargez la page qu'après avoir vu le point disparaître.